Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Операционные системы и системное программирование

ОТЧЕТ
по лабораторной работе № 3
Тема: «Взаимодействие и синхронизация процессов»

Выполнил:
ст. гр. 450502 Климова А. А.

Проверил:
Благиных В. Д.

Минск, 2026

1 ЦЕЛЬ ЛАБОРАТОРНОЙ РАБОТЫ

Изучить приемы управления дочерними процессами в ОС Linux, освоить использование системных вызовов fork(), waitpid(), setitimer(), nanosleep() и механизма сигналов SIGALRM, SIGUSR1 и SIGUSR2, а также реализовать программу, обеспечивающую создание, завершение и координацию вывода группы дочерних процессов.

Дополнительно в ходе работы требуется организовать корректное завершение процессов, динамическое хранение сведений о дочерних процессах и получение статистики, зависящей от момента прихода сигнала таймера.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Описание алгоритма выполнения работы

Программа состоит из родительского и дочерних процессов. Родительский процесс в цикле принимает команды пользователя: создание процесса (+), удаление последнего (-), вывод списка (l), удаление всех (k), завершение (q).

При создании нового дочернего процесса родитель отключает вывод у предыдущего активного дочернего процесса через SIGUSR2, затем запускает новый процесс через fork(), помечая его как активный.

Дочерний процесс устанавливает обработчики SIGALRM, SIGUSR1, SIGUSR2, запускает таймер setitimer() и в цикле поочередно записывает в пару значения (0,0) и (1,1) с паузами nanosleep(). В момент срабатывания будильника фиксируется текущее состояние пары, накапливается статистика по комбинациям 00, 01, 10, 11. Каждые 101 выборку дочерний процесс печатает статистику.

По SIGUSR1 дочерний процесс корректно завершает работу. По SIGUSR2 переключается флаг печати.

2.2 Описание основных функций и модулей

2.2.1 Главный модуль main (main.c)

Этот модуль содержит точку входа в программу и запускает логику родительского процесса.

main(void):

Устанавливает для stdout построчную буферизацию с помощью setvbuf(), чтобы сообщения родительского и дочерних процессов сразу выводились на экран построчно. После этого вызывает функцию parent_run(), которая реализует основную логику работы программы.

2.2.2 Модуль parent_process (parent_process.h, parent_process.c)

Этот модуль реализует поведение родительского процесса и управление дочерними процессами.

1) parent_run(void):

Основная функция родительского процесса. Выделяет память под динамический массив дочерних процессов, выводит список доступных команд и входит в цикл обработки пользовательского ввода. При команде “+” создает новый дочерний процесс, при “-” удаляет последний, при “l” выводит список процессов, при “k” удаляет всех дочерних процессов, при “q” завершает работу программы. Перед каждой итерацией также вызывает reap_children() для сбора уже завершившихся процессов.

2) find_child(ChildInfo *children, size_t count, pid_t pid):

Выполняет линейный поиск дочернего процесса по его PID в массиве children. Возвращает индекс найденного элемента или значение count, если процесс не найден.

3) remove_child_at(ChildInfo *children, size_t *count, size_t idx):

Удаляет элемент из массива дочерних процессов по индексу idx, сдвигая все последующие элементы влево и уменьшая текущее количество процессов.

4) set_child_output(ChildInfo *child, bool enabled):

Переключает режим вывода выбранного дочернего процесса. Если текущее состояние отличается от требуемого, родительский процесс посылает дочернему процессу сигнал SIGUSR2, после чего обновляет поле output_enabled.

5) terminate_one_child(pid_t pid):

Корректно завершает один дочерний процесс. Сначала отправляет ему сигнал SIGUSR1, затем ожидает его завершения с помощью waitpid(). Функция обрабатывает возможные прерывания системного вызова (EINTR) и ошибки ожидания.

6) kill_all_children(ChildInfo *children, size_t *count):

Удаляет все дочерние процессы, последовательно вызывая terminate_one_child() для каждого процесса из массива.

7) reap_children(ChildInfo *children, size_t *count):

Выполняет неблокирующий сбор завершившихся дочерних процессов с помощью waitpid(-1, &status, WNOHANG). Если завершился активный процесс, функция при наличии оставшихся процессов включает вывод у последнего дочернего процесса.

8) print_processes(const ChildInfo *children, size_t count):

Выводит информацию о родительском процессе и всех текущих дочерних процессах, включая их PID и состояние вывода (ON или OFF).

2.2.3 Модуль child_process (child_process.h, child_process.c)

Этот модуль содержит логику дочернего процесса, обработку сигналов и сбор статистики по состояниям пары значений.

1) child_main(bool output_enabled):

Основная функция дочернего процесса. Устанавливает обработчики сигналов SIGALRM, SIGUSR1 и SIGUSR2, инициализирует глобальные флаги и запускает бесконечный цикл работы. Во внутреннем цикле неатомарно изменяет структуру Pair, записывая значения полей с небольшими задержками через nanosleep(). После срабатывания таймера фиксирует текущее состояние пары и увеличивает соответствующий счетчик статистики. Каждые REPORT_EVERY повторений выводит статистику по комбинациям 00, 01, 10, 11. При получении SIGUSR1 завершает работу.

2) child_signal_handler(int sig):

Обработчик сигналов дочернего процесса. При SIGALRM устанавливает флаг срабатывания таймера, при SIGUSR1 устанавливает флаг завершения процесса, при SIGUSR2 переключает режим вывода статистики.

3) child_set_timer(useconds_t usec):

Настраивает однократный таймер ITIMER_REAL на заданный интервал времени с помощью setitimer(). После срабатывания таймера дочернему процессу посылается сигнал SIGALRM.

4) nanosleep_gap(long nsec):

Выполняет короткую задержку между изменениями полей структуры Pair с помощью nanosleep(). Это позволяет сделать запись значений неатомарной и получить все возможные комбинации пары в момент срабатывания будильника.

2.2.4 Конфигурационный модуль lab3_config (lab3_config.h)

Этот модуль содержит основные константы программы:

TIMER_USEC - интервал срабатывания таймера дочернего процесса в микросекундах.

WRITE_GAP_NSEC - длительность паузы между изменениями отдельных полей структуры Pair.

INITIAL_CAPACITY - начальный размер массива дочерних процессов в родительском процессе.

REPORT_EVERY - количество срабатываний таймера, после которого дочерний процесс выводит накопленную статистику.

2.2.5 Модуль общих типов lab3_types (lab3_types.h)

Этот модуль содержит общие структуры данных, используемые в программе.

Pair - структура из двух целочисленных полей first и second, которая используется дочерним процессом для формирования состояний пары.

ChildInfo - структура для хранения информации о дочернем процессе: его PID и текущего состояния вывода статистики.

2.3 Функциональная структура проекта

Проект разделен на логические модули, что упрощает его понимание, сопровождение и изменение:

- main.c: точка входа в программу, настройка буферизации вывода и запуск родительского процесса;
- parent_process.h: объявление интерфейса родительского процесса;
- parent_process.c: реализация логики родительского процесса, создание и удаление дочерних процессов, обработка команд пользователя, управление выводом и сбор завершившихся процессов;
- child_process.h: объявление интерфейса дочернего процесса;
- child_process.c: реализация логики дочернего процесса, обработка сигналов, установка таймера и сбор статистики;
- lab3_config.h: хранение основных констант программы;
- lab3_types.h: описание общих структур данных, используемых в родительском и дочернем процессах;
- Makefile: автоматизация сборки и очистки проекта.

Такое разделение позволяет независимо изменять логику родительского процесса, поведение дочернего процесса, общие типы данных и параметры эксперимента, не затрагивая остальные части программы.

3 ОПИСАНИЕ ПОРЯДКА СБОРКИ

Сборка проекта выполняется с помощью make.

Команды:

- обычная сборка: make (или make all). Компилирует исходные файлы main.c, parent_process.c, child_process.c и создает исполняемый файл lab3 в корневом каталоге проекта.

- полная пересборка: make clean && make. Удаляет объектные файлы и исполняемый файл, после чего выполняет чистую сборку проекта заново.

- очистка артефактов: make clean. Удаляет объектные файлы *.o и исполняемый файл lab3.

- результат сборки: исполняемый файл ./lab3.

Параметры компиляции:

Для сборки используется компилятор cc с флагами -W -Wall -Wextra -std=c11 -pedantic -Wno-unused-parameter -Wno-unused-variable. Это позволяет выполнять сборку в стандарте C11 с включением основных предупреждений компилятора.

Подготовка к запуску:

Перед запуском программы необходимо выполнить сборку проекта командой make. После этого программу можно запустить из корневого каталога проекта командой ./lab3.

4 РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

Вывод отсортированного окружения ./parentВ ходе тестирования программы были проверены следующие сценарии:

1) создание одного и нескольких дочерних процессов;
2) автоматическое переключение вывода статистики на последний созданный дочерний процесс;
3) вывод списка активных процессов;
4) удаление последнего дочернего процесса;
5) завершение программы с корректным завершением всех дочерних процессов.

Пример работы программы при создании трех дочерних процессов и выводе списка активных процессов:

```
[PARENT pid=6068] commands:
  +  create child
  -  remove last child
  l  list all processes
  k  kill all children
  q  quit
cmd> +
[PARENT pid=6068] created child pid=6071, total=1
cmd> [CHILD pid=6071 ppid=6068] started, output=ON
[CHILD pid=6071 ppid=6068] 00=44 01=6 10=45 11=6
[CHILD pid=6071 ppid=6068] 00=44 01=11 10=44 11=2
+
[PARENT pid=6068] created child pid=6072, total=2
cmd> [CHILD pid=6072 ppid=6068] started, output=ON
[CHILD pid=6071 ppid=6068] 00=37 01=10 10=22 11=32
[CHILD pid=6072 ppid=6068] 00=35 01=16 10=17 11=33
[CHILD pid=6071 ppid=6068] 00=68 01=10 10=10 11=13
[CHILD pid=6072 ppid=6068] 00=58 01=6 10=33 11=4
[CHILD pid=6071 ppid=6068] 00=63 01=12 10=22 11=4
+
[PARENT pid=6068] created child pid=6073, total=3
cmd> [CHILD pid=6073 ppid=6068] started, output=ON
[CHILD pid=6072 ppid=6068] 00=51 01=20 10=11 11=19
[CHILD pid=6071 ppid=6068] 00=55 01=20 10=2 11=24
l
[PARENT pid=6068] children (3):
[PARENT pid=6068]   #0 child pid=6071
[PARENT pid=6068]   #1 child pid=6072
[PARENT pid=6068]   #2 child pid=6073
```

Проверка через терминал и команду ps -ax -o pid,ppid,tty,stat,command | grep './lab3':

```
avelinaklimova@MacBook-Air-Avelina ~ % ps -ax -o pid,ppid,tty,stat,command |
grep './lab3'

 4384  2783 ttys005  Ss+  ./lab3
```

```
 4441  4384 ttys005  S+   ./lab3
 6098  4867 ttys007  R+   grep ./lab3
 5235  2783 ttys008  Ss+  ./lab3
 6068  6028 ttys009  S+   ./lab3
 6071  6068 ttys009  S+   ./lab3
 6072  6068 ttys009  S+   ./lab3
 6073  6068 ttys009  S+   ./lab3
```

После удаления последнего дочернего процесса:

```
-
[CHILD pid=6073 ppid=6068] stopped
[PARENT pid=6068] removed child pid=6073, remaining=2
cmd> [CHILD pid=6072 ppid=6068] 00=41 01=24 10=11 11=25
[CHILD pid=6071 ppid=6068] 00=62 01=12 10=14 11=13
[CHILD pid=6072 ppid=6068] 00=55 01=9 10=32 11=5
[CHILD pid=6071 ppid=6068] 00=67 01=11 10=16 11=7
[CHILD pid=6072 ppid=6068] 00=49 01=11 10=37 11=4
```

Проверка через терминал с помощью команды ps -ax -o pid,ppid,tty,stat,command | grep './lab3'

```
avelinaklimova@MacBook-Air-Avelina ~ % ps -ax -o pid,ppid,tty,stat,command |
grep './lab3'

 4384  2783 ttys005  Ss+  ./lab3
 4441  4384 ttys005  S+   ./lab3
 6143  4867 ttys007  S+   grep ./lab3
 5235  2783 ttys008  Ss+  ./lab3
 6068  6028 ttys009  S+   ./lab3
 6071  6068 ttys009  S+   ./lab3
 6072  6068 ttys009  S+   ./lab3
```

Удаление всех дочерних процессов:

```
[CHILD pid=6072 ppid=6068] 00=63 01=8 10=23 11=7
[CHILD pid=6071 ppid=6068] 00=57 01=12 10=21 11=11
[CHILD pid=6072 ppid=6068] 00=57 01=13 10=26 11=5
[CHILD pid=6071 ppid=6068] 00=65 01=14 10=18 11=4
k
[CHILD pid=6072 ppid=6068] stopped
[CHILD pid=6071 ppid=6068] stopped
[PARENT pid=6068] all children removed
```

Проверка через терминал с помощью команды ps -ax -o pid,ppid,tty,stat,command | grep './lab3'

```
avelinaklimova@MacBook-Air-Avelina ~ % ps -ax -o pid,ppid,tty,stat,command |
grep './lab3'

 4384  2783 ttys005  Ss+  ./lab3
```

```
4441  4384 ttys005  R+   ./lab3
6156  4867 ttys007  S+   grep ./lab3
5235  2783 ttys008  Ss+  ./lab3
6068  6028 ttys009  S+   ./lab3
```

По результатам работы видно, что программа корректно обрабатывает команды пользователя, создает и завершает дочерние процессы, поддерживает список активных процессов и обеспечивает управляемый вывод статистики. Дочерние процессы собирают статистику по комбинациям значений пары 00, 01, 10, 11, что подтверждает неатомарный характер изменения структуры данных.

ВЫВОД

В результате выполнения лабораторной работы была разработана программа на языке C, состоящая из родительского и дочерних процессов. Реализованы создание и завершение потомков, управление их состоянием через сигналы, динамический учет активных процессов и сбор статистики по значениям структуры, обновляемой неатомарно.

В ходе работы были закреплены навыки использования системных вызовов создания процессов, ожидания завершения, настройки таймеров и обработки сигналов. При неатомарном изменении нескольких полей структуры в момент внешнего воздействия могут наблюдаться промежуточные состояния, что и отражается в собираемой статистике.

ПРИЛОЖЕНИЕ А
(Листинг кода)

```
Файл lab3_config.h
#ifndef LAB3_CONFIG_H
#define LAB3_CONFIG_H

enum {
    TIMER_USEC = 3000,
    WRITE_GAP_NSEC = 50000,
    INITIAL_CAPACITY = 8,
    REPORT_EVERY = 101
};

#endif

Файл lab3_types.h
#ifndef LAB3_TYPES_H
#define LAB3_TYPES_H

#include <stdbool.h>
#include <sys/types.h>

typedef struct {
    int first;
    int second;
} Pair;

typedef struct {
    pid_t pid;
} ChildInfo;

#endif

Файл child_process.h
#ifndef CHILD_PROCESS_H
#define CHILD_PROCESS_H

#include <stdbool.h>

void child_main(bool output_enabled);

#endif

Файл child_process.c
#define _POSIX_C_SOURCE 200809L

#include "child_process.h"
```

```
#include "lab3_config.h"
#include "lab3_types.h"

#include <signal.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/time.h>
#include <time.h>
#include <unistd.h>

static volatile sig_atomic_t g_child_alarm = 0;
static volatile sig_atomic_t g_child_stop = 0;
static volatile sig_atomic_t g_child_print_enabled = 1;

static void child_signal_handler(int sig) {
    if (sig == SIGALRM) {
        g_child_alarm = 1;
    } else if (sig == SIGUSR1) {
        g_child_stop = 1;
    } else if (sig == SIGUSR2) {
        g_child_print_enabled = !g_child_print_enabled;
    }
}

static void child_set_timer(useconds_t usec) {
    struct itimerval timer;
    memset(&timer, 0, sizeof(timer));
    timer.it_value.tv_sec = (time_t)(usec / 1000000U);
    timer.it_value.tv_usec = (suseconds_t)(usec % 1000000U);
    if (setitimer(ITIMER_REAL, &timer, NULL) == -1) {
        perror("setitimer");
        _exit(EXIT_FAILURE);
    }
}

static void nanosleep_gap(long nsec) {
    struct timespec req;
    req.tv_sec = 0;
    req.tv_nsec = nsec;
    (void)nanosleep(&req, NULL);
}

void child_main(bool output_enabled) {
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = child_signal_handler;
    sa.sa_flags = SA_RESTART;
    sigemptyset(&sa.sa_mask);
```

```
    if (sigaction(SIGALRM, &sa, NULL) == -1 ||
        sigaction(SIGUSR1, &sa, NULL) == -1 ||
        sigaction(SIGUSR2, &sa, NULL) == -1) {
        perror("sigaction(child)");
        _exit(EXIT_FAILURE);
    }

    g_child_stop = 0;
    g_child_alarm = 0;
    g_child_print_enabled = output_enabled ? 1 : 0;

    Pair pair = {0, 0};
    unsigned long stats[2][2] = {{0, 0}, {0, 0}};
    int samples = 0;

    printf("[CHILD pid=%ld ppid=%ld] started, output=%s\n",
(long)getpid(), (long)getppid(),
           g_child_print_enabled ? "ON" : "OFF");
    fflush(stdout);

    while (!g_child_stop) {
        g_child_alarm = 0;
        child_set_timer((useconds_t)TIMER_USEC);

        while (!g_child_alarm && !g_child_stop) {
            pair.first = 0;
            nanosleep_gap((long)WRITE_GAP_NSEC);
            if (g_child_alarm || g_child_stop) {
                break;
            }

            pair.second = 0;
            nanosleep_gap((long)WRITE_GAP_NSEC);
            if (g_child_alarm || g_child_stop) {
                break;
            }

            pair.first = 1;
            nanosleep_gap((long)WRITE_GAP_NSEC);
            if (g_child_alarm || g_child_stop) {
                break;
            }

            pair.second = 1;
            nanosleep_gap((long)WRITE_GAP_NSEC);
        }

        if (g_child_stop) {
            break;
```

```
        }

        stats[pair.first][pair.second] += 1UL;
        samples += 1;

        if (samples ≥ REPORT_EVERY) {
            if (g_child_print_enabled) {
                printf("[CHILD pid=%ld ppid=%ld] 00=%lu 01=%lu 10=%lu
11=%lu\n", (long)getpid(),
                       (long)getppid(), stats[0][0], stats[0][1],
stats[1][0], stats[1][1]);
                fflush(stdout);
            }
            memset(stats, 0, sizeof(stats));
            samples = 0;
        }
    }

    printf("[CHILD pid=%ld ppid=%ld] stopped\n", (long)getpid(),
(long)getppid());
    fflush(stdout);
    _exit(EXIT_SUCCESS);
}
```

Файл parent_process.h

```
#ifndef PARENT_PROCESS_H
#define PARENT_PROCESS_H

int parent_run(void);

#endif
```

Файл parent_process.c

```
#define _POSIX_C_SOURCE 200809L

#include "parent_process.h"

#include "child_process.h"
#include "lab3_config.h"
#include "lab3_types.h"

#include <errno.h>
#include <signal.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```
#include <termios.h>
#include <unistd.h>

typedef struct {
    bool enabled;
    struct termios saved;
} TerminalMode;

static size_t find_child(ChildInfo *children, size_t count, pid_t pid)
{
    for (size_t i = 0; i < count; ++i) {
        if (children[i].pid == pid) {
            return i;
        }
    }
    return count;
}

static void remove_child_at(ChildInfo *children, size_t *count, size_t
idx) {
    for (size_t i = idx; i + 1 < *count; ++i) {
        children[i] = children[i + 1];
    }
    *count -= 1;
}

static void terminate_one_child(pid_t pid) {
    if (kill(pid, SIGUSR1) == -1 && errno != ESRCH) {
        fprintf(stderr, "[PARENT pid=%ld] failed SIGUSR1 child
pid=%ld: %s\n", (long)getpid(),
                (long)pid, strerror(errno));
    }

    int status = 0;
    while (waitpid(pid, &status, 0) == -1) {
        if (errno == EINTR) {
            continue;
        }
        if (errno != ECHILD) {
            fprintf(stderr, "[PARENT pid=%ld] waitpid(%ld) failed:
%s\n", (long)getpid(),
                    (long)pid, strerror(errno));
        }
        break;
    }
}

static void kill_all_children(ChildInfo *children, size_t *count) {
    while (*count > 0) {
        pid_t pid = children[*count - 1].pid;
```

```
        *count -= 1;
        terminate_one_child(pid);
    }
}

static void reap_children(ChildInfo *children, size_t *count) {
    while (true) {
        int status = 0;
        pid_t pid = waitpid(-1, &status, WNOHANG);
        if (pid <= 0) {
            break;
        }

        size_t idx = find_child(children, *count, pid);
        if (idx == *count) {
            continue;
        }

        remove_child_at(children, count, idx);

        printf("[PARENT pid=%ld] child pid=%ld finished,
remaining=%zu\n", (long)getpid(),
               (long)pid, *count);
    }
}

static void print_processes(const ChildInfo *children, size_t count) {
    printf("[PARENT pid=%ld] children (%zu):\n", (long)getpid(),
count);
    for (size_t i = 0; i < count; ++i) {
        printf("[PARENT pid=%ld]   #%zu child pid=%ld\n",
(long)getpid(), i,
               (long)children[i].pid);
    }
}

// переводит stdin в неканонический режим
static int enable_immediate_input(TerminalMode *mode) {
    memset(mode, 0, sizeof(*mode));

    if (!isatty(STDIN_FILENO)) {
        return 0;
    }

    if (tcgetattr(STDIN_FILENO, &mode->saved) == -1) {
        perror("tcgetattr");
        return -1;
    }

    // ICANON выключается, поэтому ввод больше не ждет Enter
```

```
    // ECHO выключается, поэтому терминал сам не печатает символ
    struct termios raw = mode->saved;
    raw.c_lflag &= (tcflag_t)~(ICANON | ECHO);
    // чтение возвращает сразу 1 символ:
    raw.c_cc[VMIN] = 1;
    raw.c_cc[VTIME] = 0;

    if (tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw) == -1) {
        perror("tcsetattr");
        return -1;
    }

    mode->enabled = true;
    return 0;
}

static void restore_terminal_mode(const TerminalMode *mode) {
    if (!mode->enabled) {
        return;
    }

    if (tcsetattr(STDIN_FILENO, TCSAFLUSH, &mode->saved) == -1) {
        perror("tcsetattr");
    }
}

static int read_command_char(char *cmd) {
    while (true) {
        ssize_t read_count = read(STDIN_FILENO, cmd, 1);
        if (read_count == 1) {
            return 1;
        }
        if (read_count == 0) {
            return 0;
        }
        if (errno == EINTR) {
            continue;
        }

        perror("read");
        return -1;
    }
}

int parent_run(void) {
    ChildInfo *children = calloc((size_t)INITIAL_CAPACITY,
sizeof(*children));
    if (children == NULL) {
        perror("calloc children");
        return EXIT_FAILURE;
```

```
    }

    size_t children_cap = INITIAL_CAPACITY;
    size_t children_count = 0;
    TerminalMode terminal_mode;

    if (enable_immediate_input(&terminal_mode) == -1) {
        free(children);
        return EXIT_FAILURE;
    }

    printf("[PARENT pid=%ld] commands:\n", (long)getpid());
    printf("  +  create child\n");
    printf("  -  remove last child\n");
    printf("  l  list all processes\n");
    printf("  k  kill all children\n");
    printf("  q  quit\n");

    bool running = true;

    while (running) {
        reap_children(children, &children_count);

        printf("cmd> ");
        fflush(stdout);

        char cmd = '\0';
        int read_status = read_command_char(&cmd);
        if (read_status <= 0) {
            printf("\n");
            break;
        }

        if (terminal_mode.enabled) {
            if (cmd != '\n' && cmd != '\r') {
                putchar(cmd);
            }
            printf("\n");
        }

        if (cmd == '+') {
            if (children_count == children_cap) {
                size_t new_cap = children_cap * 2;
                ChildInfo *new_children = realloc(children, new_cap *
sizeof(*new_children));
                if (new_children == NULL) {
                    perror("realloc children");
                    continue;
                }
                children = new_children;
```

```
                children_cap = new_cap;
            }

            pid_t pid = fork();
            if (pid == -1) {
                perror("fork");
                continue;
            }
            if (pid == 0) {
                child_main(true);
            }

            children[children_count].pid = pid;
            children_count += 1;

            printf("[PARENT pid=%ld] created child pid=%ld,
total=%zu\n", (long)getpid(),
                    (long)pid, children_count);
        } else if (cmd == '-') {
            if (children_count == 0) {
                printf("[PARENT pid=%ld] no children to remove\n",
(long)getpid());
                continue;
            }

            pid_t pid = children[children_count - 1].pid;
            children_count -= 1;
            terminate_one_child(pid);

            printf("[PARENT pid=%ld] removed child pid=%ld,
remaining=%zu\n", (long)getpid(),
                    (long)pid, children_count);
        } else if (cmd == 'l' || cmd == 'L') {
            print_processes(children, children_count);
        } else if (cmd == 'k' || cmd == 'K') {
            kill_all_children(children, &children_count);
            printf("[PARENT pid=%ld] all children removed\n",
(long)getpid());
        } else if (cmd == 'q' || cmd == 'Q') {
            kill_all_children(children, &children_count);
            running = false;
            printf("[PARENT pid=%ld] quit\n", (long)getpid());
        } else if (cmd == '\n') {
            continue;
        } else {
            printf("[PARENT pid=%ld] unknown command: %c\n",
(long)getpid(), cmd);
        }
    }
```

```
    if (children_count > 0) {
        kill_all_children(children, &children_count);
    }

    restore_terminal_mode(&terminal_mode);
    free(children);
    return EXIT_SUCCESS;
}
```

Файл main.c

```
#define _POSIX_C_SOURCE 200809L

#include "parent_process.h"

#include <stdio.h>

int main(void) {
    setvbuf(stdout, NULL, _IOLBF, 0);
    return parent_run();
}
```